University of Eastern Finland

Advanced Project in Robotics

Waste-Sorting Robot

Project Report

Authors: Jan Sistonen and Jermu Roivanen
Course: Advanced Project in Robotics
Year: 2026

Abstract

This project developed a prototype waste-sorting robot that combines computer vision, embedded control, and a custom linear transport mechanism. A Raspberry Pi 5 runs a fine-tuned YOLO object-detection model and maps the detected object label to one of five waste categories: plastic, organic waste, metal, cardboard, or mixed waste. A NEMA 17 stepper motor moves a sorting basket along an aluminium frame using a timing belt, while a servo-operated hatch releases the item at the selected position. An ultrasonic sensor is used for homing and position reference. The final prototype demonstrated the complete Sense–Think–Act sequence from image capture to mechanical sorting. The most significant development effort was required in the mechanical design, where several 3D-printed pulley, motor-mount, and basket-carriage solutions were tested before a stable configuration was achieved.

Contents

1 Introduction

We wanted to choose an interesting, yet meaningful topic for our project. After considering several options, we selected a challenging and necessary topic for our project: waste sorting. Scientific studies show that people often put waste in the incorrect bins in public areas. Human error decreases the quality of recycled materials. In a recent field study conducted at four public locations in two Austrian cities, only approximately 20 percent of recyclable materials were disposed of in the correct waste fraction (Kladnik, V. et al., 2025). Other studies conducted at university campuses and public events have reported waste-stream contamination rates of approximately 23–44percent (Hottle, T. et al., 2015). Incorrect sorting reduces the quality and value of separately collected materials, increases processing costs, and may result in contaminated bins or loads being treated as residual waste instead of being recycled.

Because of this, it is on demand to sort waste automatically at the source, not only at the waste management facility. Our robot-assisted waste sorting solution would resolve this issue by handling the sorting process on behalf of the users.

The largest challenge in this project is to identify the different types of waste correctly. Other challenges include the mechanical design and the integration of hardware and software.

2 Project Objectives and System Concept

The primary objective was to build a functional prototype that accepts one waste item at a time, identifies the item using a camera, moves it to the correct waste position, releases it, and returns to a known starting position. The system was designed around three functional layers:

- **Perception:** image acquisition with a USB camera and waste classification with a fine-tuned YOLO model;
- **Control:** category mapping, homing, target-position selection, and actuator sequencing on a Raspberry Pi 5;
- **Actuation:** linear basket movement using a stepper motor and timing belt, followed by hatch operation using a servo motor.

2.1 Sense–Think–Act Architecture

Figure 1 summarises the system architecture.

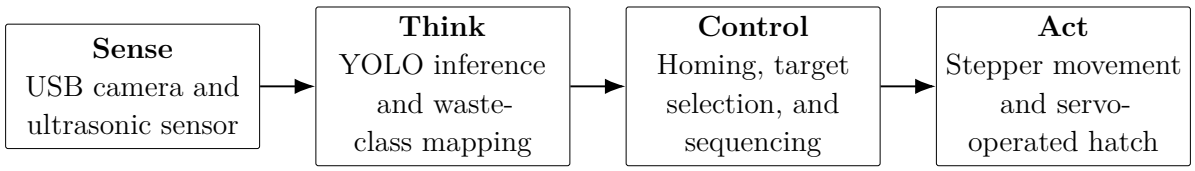

Figure 1: Functional architecture of the waste-sorting prototype.

2.2 Operating Sequence

The implemented operating sequence was as follows:

1. At program start, the basket performs a homing movement until the ultrasonic sensor detects the reference position beneath the camera.
2. The user places one item in the sorting basket.
3. The USB camera captures an image, and the YOLO model predicts an object label and confidence score.
4. The software maps the predicted label to a waste category and calculates the corresponding basket position.
5. The Raspberry Pi sends STEP and DIR signals to the stepper-motor driver until the target position is reached.
6. The servo motor opens the bottom hatch and releases the item.
7. The hatch closes, and the basket returns to the home position.

3 Hardware Design

Table 1 lists the main hardware components used in the prototype.

Table 1: Main hardware components and their roles.

Component	Function
Raspberry Pi 5	Central controller for image processing, sensor input, actuator commands, and overall program logic.
USB webcam	Captures the waste item for object detection.
Ultrasonic sensor	Provides a reference for homing and basket-position detection.
NEMA 17 stepper motor	Produces controlled rotary motion for the linear basket drive.
A4988 stepper driver	Receives STEP and DIR signals and regulates motor-winding current.
Servo motor	Opens and closes the hatch on the bottom of the basket.
Timing belt and pulleys	Convert stepper-motor rotation into linear basket movement.
4040 aluminium profiles	Form the structural frame and guide rails of the prototype.
3D-printed parts	Provide custom motor mounts, pulley supports, belt attachments, and basket-carriage components.
Power supplies	A separate 5 V, 5 A supply powers the Raspberry Pi. The stepper subsystem uses a 3-cell 11.1 V, 1200 mA h, 20C Li-Po battery.

3.1 A4988 Current-Limit Calibration

The NEMA 17 stepper motor was controlled through an A4988 driver connected to the Raspberry Pi GPIO pins. The driver receives STEP and DIR signals and limits the current supplied to the motor windings. Correct current-limit adjustment is important because an excessively high setting can overheat the driver or motor, whereas a setting that is too low can reduce torque and cause missed steps.

According to the A4988 manufacturer datasheet, the maximum current trip level is (Allegro MicroSystems 2022)

$$I_{\text{TripMAX}} = \frac{V_{\text{REF}}}{8R_S}, \tag{1}$$

which can be rearranged as

$$V_{\text{REF}} = 8R_S I_{\text{TripMAX}}, \tag{2}$$

where R_S is the current-sense resistance on the driver module and I_{TripMAX} is the desired maximum phase current. Because A4988 carrier boards can use different sense-resistor values, the resistor marking on the actual module must be checked before calculating the target voltage.

The reference voltage was measured between the VREF adjustment point and ground using a multimeter. The onboard potentiometer was then adjusted in small increments until the calculated value was reached. The setting was verified through practical motion tests while monitoring motor torque and driver temperature.

4 Perception and Software

4.1 Machine-Learning Model and Dataset

A custom waste-recognition model was created by fine-tuning a pre-trained YOLO object-detection model. Project-specific images were collected and annotated using Roboflow. A naming convention was used to connect object-level labels to material categories. For example, the label `bio_banana` contains the prefix `bio`, which the Python program maps to the organic-waste position.

The model therefore performs object recognition, while a separate software mapping layer converts the predicted label into one of the five sorting categories:

- plastic;
- organic waste;
- metal;
- cardboard;
- mixed waste.

The model was first tested on a personal computer before being deployed to the Raspberry Pi. OpenCV was used for camera access and image handling. The Raspberry Pi 5 provided sufficient performance for the prototype's inference and control requirements.

4.2 Software Environment and Interfaces

The system ran on 64-bit Raspberry Pi OS and was implemented in Python. Development and system management were performed remotely over SSH. The main interfaces were:

- **Camera:** USB connection to the Raspberry Pi;
- **Ultrasonic sensor:** GPIO trigger and echo signals;
- **Stepper driver:** GPIO STEP and DIR outputs, with optional microstepping configuration pins;
- **Servo motor:** PWM control signal from the Raspberry Pi;
- **Hardware abstraction:** the `gpiozero` Python library for GPIO control.

The prototype was intended for indoor operation. Consistent illumination was important because strong shadows and changing backgrounds could reduce classification reliability.

5 Implementation Process

The project was completed in five main phases.

Phase 1: Initial software prototype. A dataset was created, the YOLO model was trained, and object detection was tested with OpenCV on an Ubuntu computer.

Phase 2: Basic Raspberry Pi control. GPIO access was tested with simple components, followed by servo-motor control using the `gpiozero` library.

Phase 3: Stepper-motor integration. The NEMA 17 motor and A4988 driver were installed, the current limit was calculated and calibrated, and basic motion routines were implemented.

Phase 4: System integration. The camera, trained model, sensor inputs, stepper movement, servo hatch, and control logic were combined into one end-to-end program.

Phase 5: Physical assembly. The frame was built from 4040 aluminium profiles. Custom mechanical components were designed in CAD, 3D-printed, tested, and revised during assembly.

6 Mechanical Design Iterations

The software pipeline functioned comparatively early in the project, but the mechanical system required several redesigns. The main difficulty was converting the stepper motor's rotary motion into smooth and repeatable linear movement while keeping the timing belt aligned and sufficiently tensioned.

Early versions used simple printed idlers and a basket supported directly by small printed wheels. These concepts demonstrated the basic principle but produced belt misalignment, excessive friction, or insufficient structural stiffness. Later versions introduced revised motor mounts, dual-idler arrangements, stronger corner brackets, and a more controlled belt connection to the moving carriage. Several parts were intentionally treated as rapid prototypes: they were printed, tested under load, evaluated, and either modified or discarded.

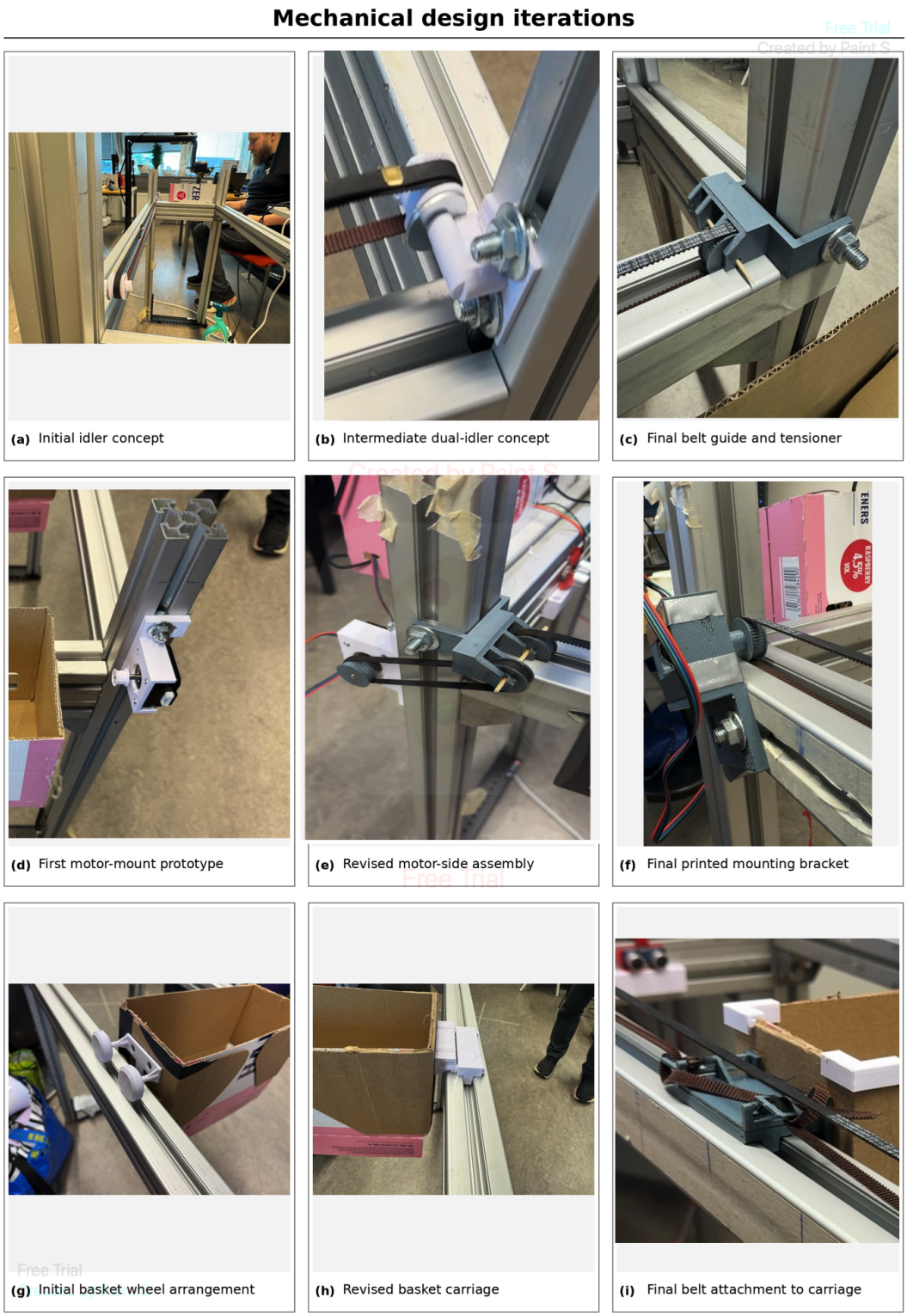

Figure 2: Selected mechanical design iterations. The top row shows the development of the belt-routing and tensioning concepts, the middle row shows motor-side mounting solutions, and the bottom row shows the development of the basket carriage and belt attachment.

The final arrangement provided a more stable belt path and a clearer separation between the fixed motor assembly and the moving basket. The iterative process also demonstrated the practical value of additive manufacturing in robotics: custom interfaces could be produced quickly, but printed parts still required careful attention to load direction, stiffness, friction, tolerances, and print orientation.

7 Evaluation and Results

The completed prototype demonstrated the intended end-to-end sequence: homing, image capture, classification, movement to the selected position, hatch operation, and return to the starting position. The system therefore met the core functional objective of integrating perception, embedded control, and mechanical actuation in one working robot prototype.

The strongest aspect of the project was the successful integration of multiple subsystems. The camera, machine-learning model, GPIO interfaces, stepper driver, sensor, servo, and custom mechanics operated as a unified system. The project also provided practical experience in troubleshooting electrical power, calibrating a motor driver, designing parts for 3D printing, and translating a conceptual architecture into a physical prototype.

The evaluation was nevertheless limited by the available project time. A formal classification-accuracy test was not completed, and mechanical testing did not cover long-duration operation, a wide range of loads, or repeated-cycle reliability. The current software also executes most actions sequentially. Consequently, the video stream pauses while blocking motor functions are running, which is inconvenient during remote debugging even though continuous video would not be essential in a closed final product.

Based on the completed objectives, technical difficulty, and amount of independent system integration, the team assesses the project as meeting the criteria for the highest course grade. This self-assessment should be considered together with the limitations described above.

8 Limitations and Future Work

The following improvements would make the prototype more reliable and closer to a deployable robotic system:

- **ROS 2 architecture:** divide the current single Python program into separate perception, motion-control, sensor, and system-state nodes;
- **Non-blocking execution:** use asynchronous execution, threading, or ROS 2 executors so that image capture and status monitoring continue during movement;
- **Safety and state control:** add explicit state-machine logic, timeout handling, emergency stopping, and protection against conflicting actuator commands;
- **Improved positioning:** add limit switches, an encoder, or other feedback to detect missed steps and improve repeatability;

- **Mechanical refinement:** replace temporary cardboard structures with a rigid enclosed basket and protected belt drive;
- **Perception evaluation:** collect a larger and more balanced dataset and report precision, recall, confusion matrices, and inference latency;
- **Alternative classification pipeline:** combine a general object detector with a separate material- or waste-category mapping component. Public datasets such as TACO provide diverse litter annotations but are not directly organised around the exact disposal categories required by this prototype (Proença and Simões 2020). A language model or a deterministic database could be used to map detected object names to local waste categories, reducing the amount of category-specific image annotation required.

9 Project Resources

The project source code and a video demonstration of the completed prototype are available through the following links:

- **Demonstration video:** Open the project demonstration video
- **Source code repository:** `https://github.com/jansistonen/waste-sorter`

10 Contribution and Working Hours

Table 2: Team contributions and working hours.

Member	Hours	Main responsibilities
Jan Sistonen	*180*	*3D modeling and printing, software, mechanical design, testing, documentation, and shared integration tasks.*
Jermu Roivanen	*180*	*Dataset, model training, mechanical parts, software, testing, documentation, and shared integration tasks.*

References

Allegro MicroSystems (2022). *A4988: DMOS Microstepping Driver with Translator and Overcurrent Protection*. Datasheet, Revision 8. Allegro MicroSystems. URL: `https://www.allegromicro.com/~/media/files/datasheets/a4988-datasheet.pdf` (visited on 07/31/2026).

Proença, Pedro F. and Pedro Simões (2020). "TACO: Trash Annotations in Context for Litter Detection". In: arXiv: `2003.06975 [cs.CV]`.